

Mujoco MPC build notes

Select Tags, select v0.0.1

Download as zip and unzip to verify you have the correct version

brew install ninja zlib

```
cd mujoco_mpc  
mkdir build  
cd build
```

Remove --gc-sections is a GNU/ELF-style linker flag.
--gc-sections error

Replace MukocoLinkOptions.cmake with this:

```
# Copyright 2021 DeepMind Technologies Limited  
#  
# Licensed under the Apache License, Version 2.0 (the "License");  
# you may not use this file except in compliance with the License.  
# You may obtain a copy of the License at  
#  
#   https://www.apache.org/licenses/LICENSE-2.0  
#  
# Unless required by applicable law or agreed to in writing, software  
# distributed under the License is distributed on an "AS IS" BASIS,  
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
# See the License for the specific language governing permissions and  
# limitations under the License.  
  
include(CheckCSourceCompiles)  
  
# Gets the appropriate linker options for building MuJoCo, based on features available on the  
# linker.  
function(get_mujoco_extra_link_options OUTPUT_VAR)  
  if(MSVC)  
    set(EXTRA_LINK_OPTIONS /OPT:REF /OPT:ICF=5)  
  else()  
    set(EXTRA_LINK_OPTIONS)  
  
    if(WIN32)  
      set(CMAKE_REQUIRED_FLAGS "-fuse-lld=lld-link")  
      check_c_source_compiles("int main() { return 0; }" SUPPORTS_LLD)  
      if(SUPPORTS_LLD)  
        set(EXTRA_LINK_OPTIONS  
          ${EXTRA_LINK_OPTIONS}  
          -fuse-lld=lld-link  
          -Wl,/OPT:REF  
          -Wl,/OPT:ICF  
        )  
      else()  
        # Fallback to the default linker options for Windows  
        set(EXTRA_LINK_OPTIONS /OPT:REF /OPT:ICF=5)  
      end()  
    else()  
      # Fallback to the default linker options for Linux  
      set(EXTRA_LINK_OPTIONS -Wl,-z,relro -Wl,-z,now)  
    end()  
  end()  
  set(${OUTPUT_VAR} ${EXTRA_LINK_OPTIONS} PARENT_SCOPE)  
endfunction()
```

```

    )
endif()
else()
# Prefer LLD or gold where available on non-Windows platforms.
# On macOS, do not force GNU-style section GC flags; use -dead_strip instead.
if(NOT APPLE)
    set(CMAKE_REQUIRED_FLAGS "-fuse-lld=lld")
    check_c_source_compiles("int main() { return 0; }" SUPPORTS_LLD)
    if(SUPPORTS_LLD)
        set(EXTRA_LINK_OPTIONS ${EXTRA_LINK_OPTIONS} -fuse-lld=lld)
    else()
        set(CMAKE_REQUIRED_FLAGS "-fuse-lld=gold")
        check_c_source_compiles("int main() { return 0; }" SUPPORTS_GOLD)
        if(SUPPORTS_GOLD)
            set(EXTRA_LINK_OPTIONS ${EXTRA_LINK_OPTIONS} -fuse-lld=gold)
        endif()
    endif()
endif()

if(APPLE)
    set(CMAKE_REQUIRED_FLAGS "${EXTRA_LINK_OPTIONS} -Wl,-dead_strip")
    check_c_source_compiles("int main() { return 0; }" SUPPORTS_DEAD_STRIP)
    if(SUPPORTS_DEAD_STRIP)
        set(EXTRA_LINK_OPTIONS ${EXTRA_LINK_OPTIONS} -Wl,-dead_strip)
    endif()
else()
    set(CMAKE_REQUIRED_FLAGS "${EXTRA_LINK_OPTIONS} -Wl,--gc-sections")
    check_c_source_compiles("int main() { return 0; }" SUPPORTS_GC_SECTIONS)
    if(SUPPORTS_GC_SECTIONS)
        set(EXTRA_LINK_OPTIONS ${EXTRA_LINK_OPTIONS} -Wl,--gc-sections)
    else()
        set(CMAKE_REQUIRED_FLAGS "${EXTRA_LINK_OPTIONS} -Wl,-dead_strip")
        check_c_source_compiles("int main() { return 0; }" SUPPORTS_DEAD_STRIP)
        if(SUPPORTS_DEAD_STRIP)
            set(EXTRA_LINK_OPTIONS ${EXTRA_LINK_OPTIONS} -Wl,-dead_strip)
        endif()
    endif()
endif()
endif()

set("${OUTPUT_VAR}"
    ${EXTRA_LINK_OPTIONS}
    PARENT_SCOPE
)
endfunction()

```

add _

rm -rf build

cmake -S . -B build -G Ninja -DCMAKE_POLICY_VERSION_MINIMUM=3.5

cmake --build build

this auto-creates the build directory.
once done ./build/bin/mjpc

The source code has been modified and added as a subdirectory in hw2. This isn't running RL, this is calling the controller with parameters.

Where are the weights set?

app.cc

```
double w0 = 0.0, w1 = 0.0, w2 = 0.0, w3 = 0.0;

if (task_name == "Quadruped Flat") {
    // set task weights
    sim->agent->ActiveTask()->weight[1] = w0;
    sim->agent->ActiveTask()->weight[2] = w1;

    // set task parameters
    sim->agent->ActiveTask()->parameters[5] = w2;
    sim->agent->ActiveTask()->parameters[6] = w3;
} else if (task_name == "Hand") {
    // set task weights
    sim->agent->ActiveTask()->weight[0] = w0;
    sim->agent->ActiveTask()->weight[1] = w1;
    sim->agent->ActiveTask()->weight[2] = w2;
    sim->agent->ActiveTask()->weight[3] = w3;
}

std::string video_fname = "";
// construct video file name
if (task_name == "Quadruped Flat") {
    video_fname += "quadruped";
} else if (task_name == "Hand") {
    video_fname += "hand";
}
video_fname += "_planH_" + std::to_string(absl::GetFlag(FLAGS_horizon));
video_fname += "_w0_" + std::to_string(w0);
video_fname += "_w1_" + std::to_string(w1);
video_fname += "_w2_" + std::to_string(w2);
video_fname += "_w3_" + std::to_string(w3);

// planning threads
```

Modify thread count to 1.

RL didnt work for control until 2014.

Difference between Planning and RL. Planning fast. RL slow because of exploration.
The code in agent.cc shows the planner call.

agent.cc

```

cost_ = 0.0;

// counter
count_ = 0;

mju::strcpy_arr(this->planner_names_, kPlannerNames);

// max threads
max_threads_ = 1; //std::max(1, NumAvailableHardwareThreads() - 3);
}

// allocate memory
void iLQGBackwardPass::Allocate() {

```

mjpc/planners/ilqg/planner.cc is the top-level planner loop

mjpc/planners/ilqg/backward_pass.cc is the Riccati / gain-computation step

That matches the build, which includes both planners/ilqg/planner.cc and planners/ilqg/backward_pass.cc, and your task selects planner index 2, which is the iLQG planner in the registry.

The main flow is:

In planner.cc

- iLQGPlanner::Initialize(...) sets dimensions, reads settings like ilqg_num_rollouts, and prepares the planner state.
- Allocate() allocates trajectories, model derivatives, cost derivatives, backward-pass buffers, policy objects, and box-QP storage.
- Reset() zeros everything and resets the nominal and candidate policies.
- OptimizePolicy(...) is the main entry point. It calls:
 - NominalTrajectory(...)
 - then Iteration(...)
 - NominalTrajectory(...) does the rollout phase:
 - copies the current policy into candidate policies
 - runs feedback rollouts in parallel
 - picks the best rollout
 - updates the nominal trajectory and feedback scaling

So planner.cc is basically the outer optimization loop: roll out candidates, choose the best, then iterate.

In backward_pass.cc

- iLQGBackwardPass::Allocate(...) allocates value-function and Q-function derivative buffers:
 - V_x , V_{xx}
 - Q_x , Q_u
 - Q_{xx} , Q_{xu} , Q_{uu}
 - Reset(...) zeros them and resets regularization

- RiccatiStep(...) is the core per-time-step iLQG math:
- builds local Q derivatives from dynamics and cost derivatives
- regularizes Quu
- solves for:
- dut = feedforward action update
- Kt = feedback gain
- updates the value function approximation Vx, Vxx
- Riccati(...) runs that backward over the horizon

So backward_pass.cc is the inner math that computes the control update from derivatives.

A good mental map is:

- planner.cc = algorithm orchestration
- backward_pass.cc = backward dynamic-programming step
- policy.cc = policy representation / storage
- cost_derivatives.cc and model_derivatives.cc = derivative inputs to iLQG
- task_flat.xml / quadrupe.cc = task cost being optimized

If you want to step through it in the debugger, the best breakpoints are

model-based nonlinear trajectory optimization algorithm

LQR is Linear Quadratic Regulator.

It is a classic control algorithm that finds the best control law for a system when:

- the dynamics are linear
- the objective is a quadratic cost

The setup

You assume dynamics like:

$$x_{t+1} = A x_t + B u_t$$

where:

- x_t = state
- u_t = control
- A, B = system matrices

And you want to minimize a cost like:

$$J = \sum_{t=0}^{T-1} (x_t^T Q x_t + u_t^T R u_t) + x_T^T Q_f x_T$$

where:

- Q penalizes bad states
- R penalizes large controls
- Q_f is terminal state cost

And you want to minimize a cost like:

$$J = \sum_{t=0}^{T-1} (x_t^T Q x_t + u_t^T R u_t) + x_T^T Q_f x_T$$

where:

- Q penalizes bad states
- R penalizes large controls
- Q_f is terminal state cost

So LQR answers:

what control should I apply to keep the state near the target while not using excessive effort?

The solution

LQR gives a feedback law:

$$u_t = -K x_t$$

or around a target state:

$$u_t = -K(x_t - x^*)$$

where K is the gain matrix computed from the Riccati equation.

That means:

- if the state deviates, control pushes it back
- the amount of correction is chosen optimally under the cost

Intuition

LQR balances two things:

- keep the system near the desired state
- avoid using too much control effort

If you make Q bigger:

- it cares more about state accuracy

If you make R bigger:

- it cares more about cheap/small control

Example

For a balancing robot or inverted pendulum:

- state might be angle and angular velocity
- control might be motor torque

LQR computes torque as a linear function of how tilted and how fast it is falling.

So if it leans left, torque pushes it right.

Why it is important

LQR is important because it is:

- mathematically clean
- efficient
- stable for many linear systems
- the building block for more advanced methods like iLQG and DDP

Difference from iLQG

- LQR: exact solution for linear dynamics + quadratic cost
- iLQG: repeatedly approximates a nonlinear problem as LQR-like and solves it iteratively

So iLQG is like “LQR, but applied locally over and over for nonlinear systems.” with Taylor series approximation to make the nonlinear part look linear. With small steps we can use this approximation.

Continuous-time version

There is also continuous-time LQR, with dynamics:

$$\dot{x} = A x + B u$$

and similar quadratic cost.

Same idea, different equations.

One-line summary

LQR is an optimal feedback controller for linear systems that minimizes a quadratic tradeoff between state error and control effort.